

OmniCache: 以内存为中心的KV管理与推理加速

演讲人: Ken Zhang

华为 / 高级技术专家

AiCon

全球人工智能开发与应用大会

目录

挑战：HBM成为推理瓶颈

从 KV 存算分离到以内存为中心

KV 传输掩盖与Kernel直接访存

PageAttention or Not?

KVM 与推理引擎解耦和服务化部署

KV fabric 实现全局共享



问题与挑战

以 HBM 为中心的 KV 管理架构面临的挑战

挑战：HBM 成为推理瓶颈



- KV Block Pool 启动时固定分配，无法动态回收
- MoE 专家负载不均 → 激活显存飙升 → OOM
- 被迫保守设置 max-num-batch-tokens 上限，硬件利用率不足
- 718B模型PD分离导致ping-pong效应，序列再减半，APC接近0

HBM KV静态预分配



- PagedAttention 将 block 分散在 HBM 各处
- Block Table + Slot Mapping 增加每个算子复杂度
- 框架的大部分时间花费在了 Attention Metadata构建和维护上
- 新 Attention 算法开发门槛高

地址非连续



- 无 head 维度的 latent KV 无法按 TP rank 切分
- TP16 → 16 份完全相同的 KV 副本
- Context Parallel 引入 all-gather 通信开销

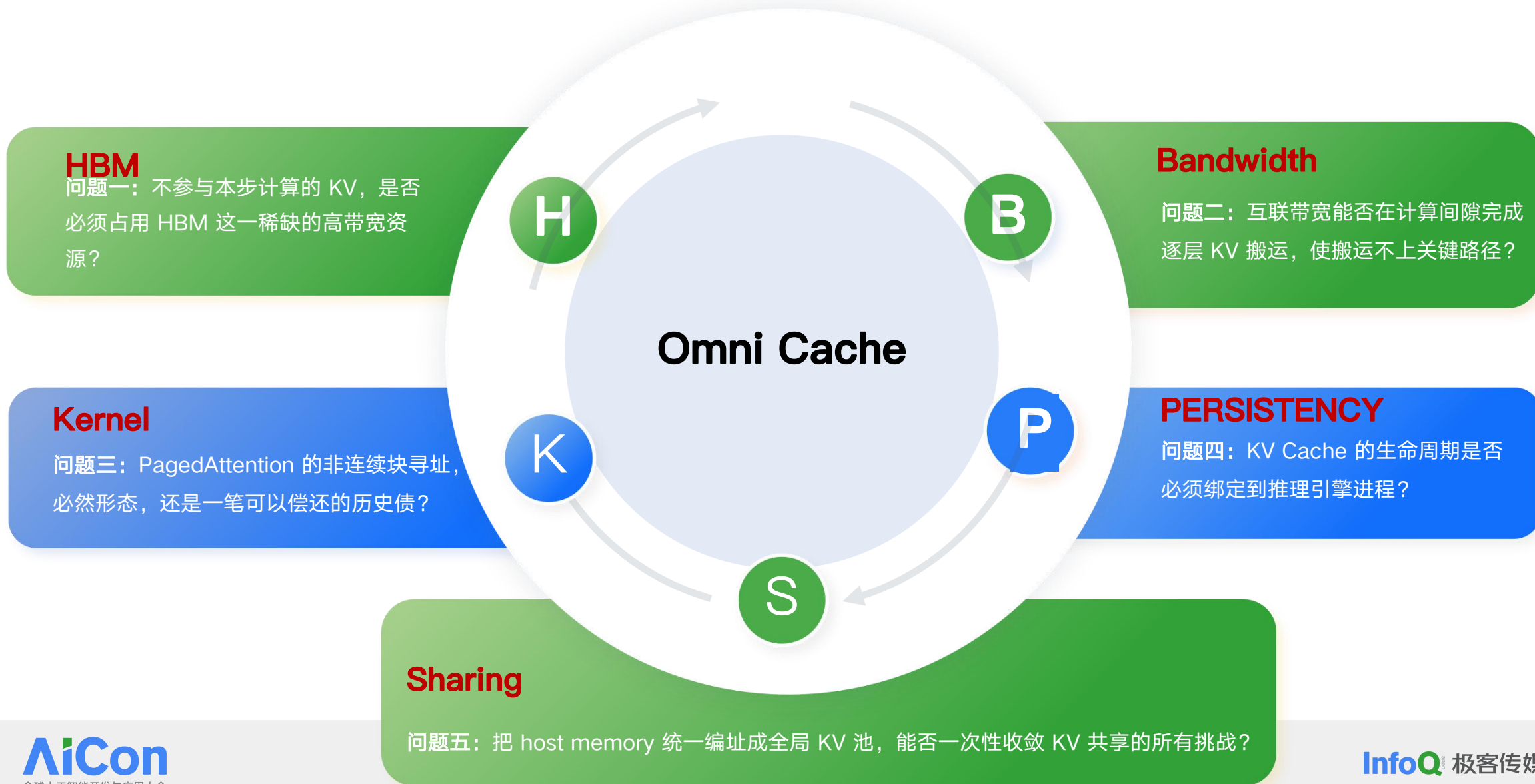
MLA 在 TP 下冗余



- 每个推理实例独立维护 KV Pool
- APC 仅限单实例 HBM (6-10 GB)
- PD 分离每次全量搬运 KV
 - Prefill KV在D侧冗余
 - Decode KV在Prefill无法访问

KV跨实例不共享

五个第一性原理的拷问？



问题一 | 不参与本步计算的 KV，是否必须占用 HBM 这一稀缺的高带宽资源？

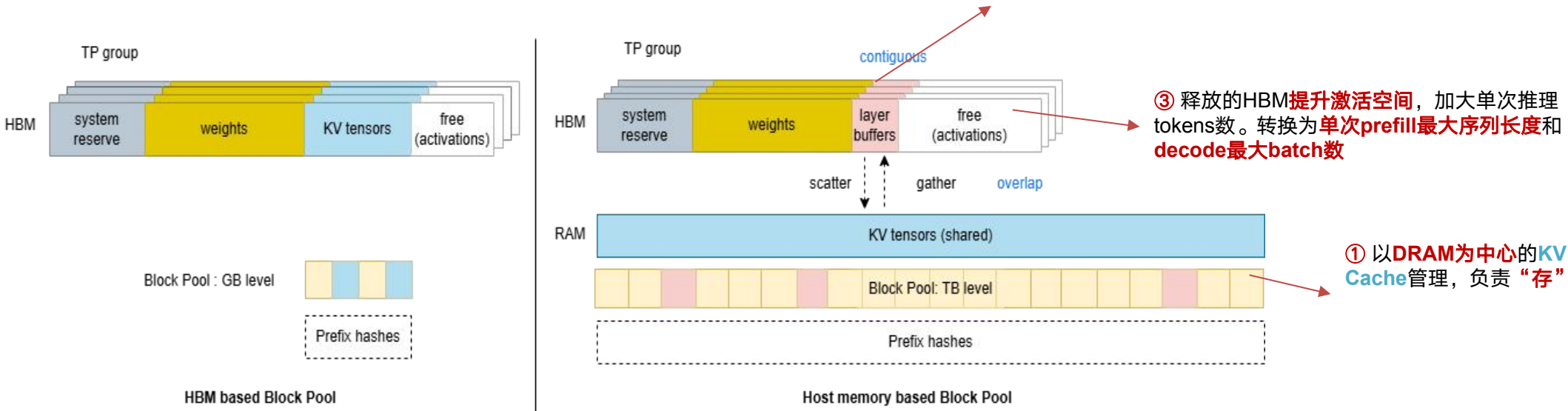
01

Omni Cache以内存为中心的架构

Omni Cache 原理与收益来源

核心定义

KV Buffer: 参与当前attention计算 **KV Cache:** 不参与，或部分参与



Host Memory Block Pool

- TB 级容量 (对比 GB 级 HBM)
- 所有 TP rank 和实例共享同一份 KV
- 彻底消除 MLA 冗余
- 原生 APC 支持 — block 数量提升 100×

HBM 连续地址缓冲区

- 每请求连续虚拟地址空间
- 两步式 VMM: 先预留 VA, 再按需绑定物理页
- 无需 Block Table 和 Slot Mapping
- 大幅简化 Attention 算子开发

逐层异步搬运&跨机按需访问

- D2H scatter 与下一层计算重叠
 - H2D gather 与 Attention 计算重叠
 - 自研 Device 端 kernel: gather 117 GB/s vs 60GB/s
 - 节点间RDMA/HCCS单边访问26GB / 104GB
- 目标: 对推理性能零影响

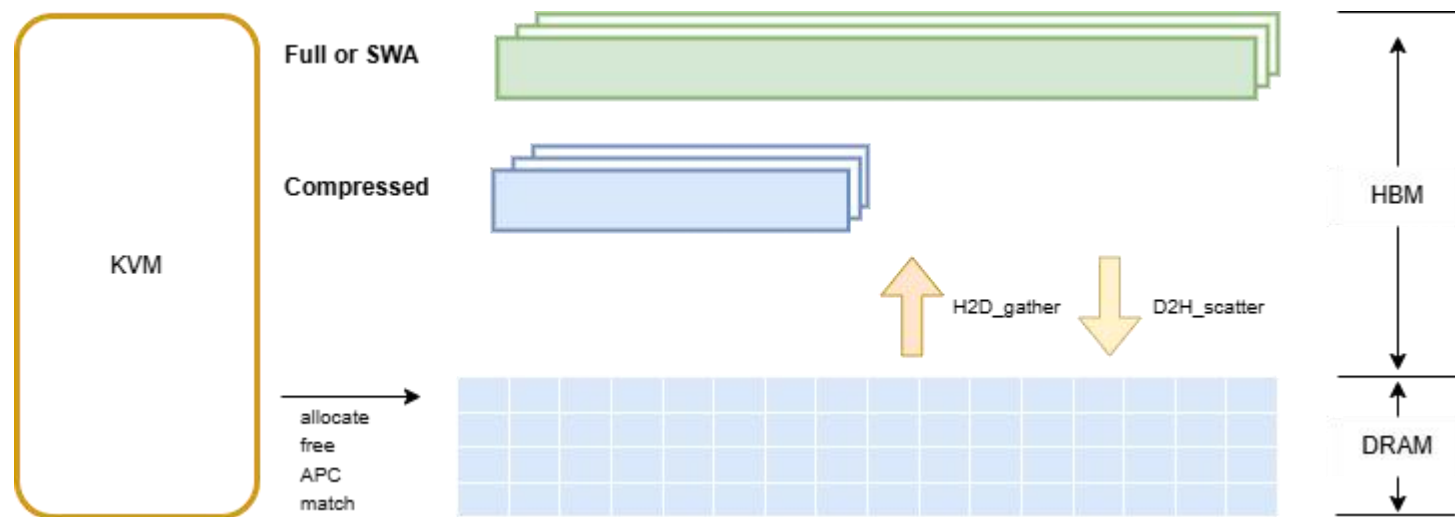
效果: 非计算KV 零HBM占用 · DSA 80%+ KV 卸载 · DSA Decode 并发提升 4× · APC 容量提升 10~100× · MLA 零冗余

问题二 | 互联带宽能否在计算间隙完成逐层 KV 搬运，使搬运不上关键路径？

02

KV Buffer与Kernel Direct Access

■ Prefill 单层KV Buffer



N层 KV Buffer + Lockstep Gather/Scatter

以N=1单层KV Buffer为例：layer l 进入 attention 前由 gather 从 host pool 装入该层 KV（含 APC 命中前缀或 chunked prefill部分KV），attention 算完立即由 scatter 写回 host pool。

效果：HBM 中 KV 从 L 层压成 1 层，更多的激活空间，更大的单次计算budgets，更高的算力利用率

TopKSelection&TBO支撑更长上下文：稀疏注意力的天然卸载机会

基于TopK的稀疏KV → 大部分 KV 可留在 Host，不占 HBM

现代模型收敛到「四区」KV 结构的组合（ShadowKV，Quest，DSA，DeepSeek V4）：

Sink 区

首批 token 始终保留
(Attention Sink)

滑动窗口区

最近 W 个 token 始终在
HBM
(局部上下文)

选择区

占 KV 总量大部分 — 每步
仅稀疏选取 top-K

Recurrent

固定空间 + Snapshots
Mamba, DeltaNet, GDN

↓ 选择区 = 75 – 80% 的 KV 存储量，稀疏访问 → 天然适合 Host 内存卸载

- 稀疏注意力将大部分 KV 留在 Host — 每步仅加载 top-K 选择 → HBM 占用极低
- C4 缓冲区复用：4× 缓存实现 90%+ 命中率 → 每步仅约 75 个 KV 向量需搬运
- 三阶段 Batch 重叠（TBO）：SWA → 缓存命中 → 缓存未命中的 mini-batch 掩盖搬运延迟

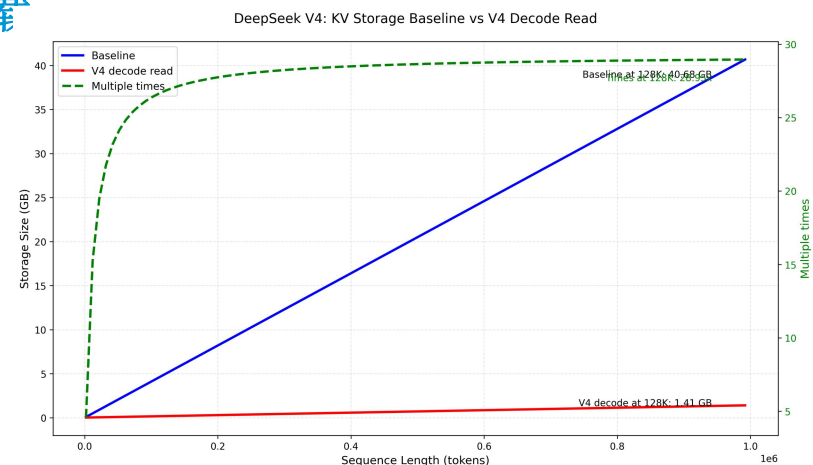
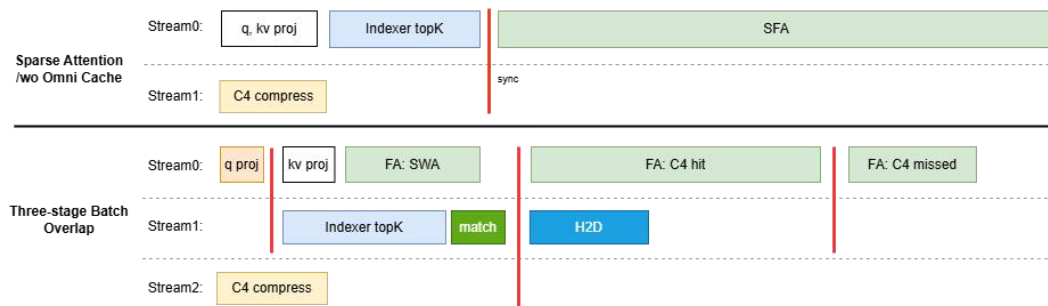


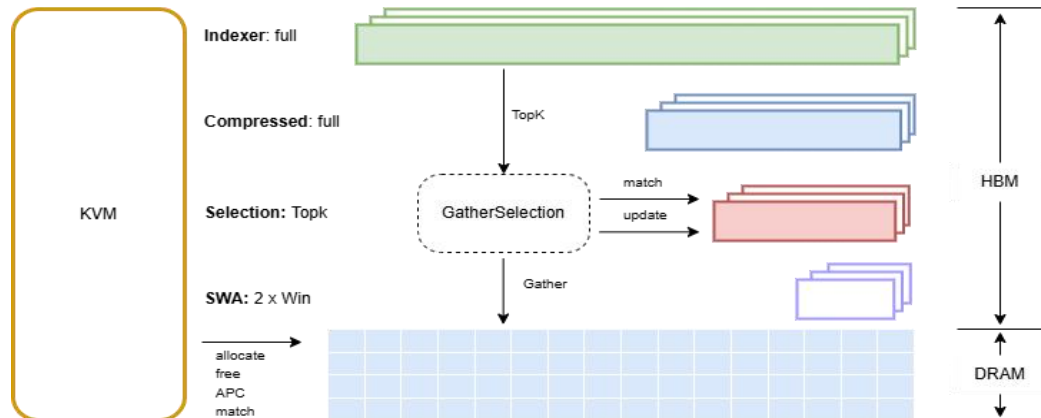
Figure 3: Decode phase KV read volume reduction factor of DeepSeek V4 (with SWA) compared to standard MLA normalized to 512-dimensional KV head dimension. The reduction factor grows with sequence length, reaching approximately 25x at 64K and approaching a theoretical asymptote of 29x for very long sequences, demonstrating the dramatic memory bandwidth savings enabled by sparse KV access via Lightning indexing.

Decode基于统一寻址Hybrid KV Buffer策略

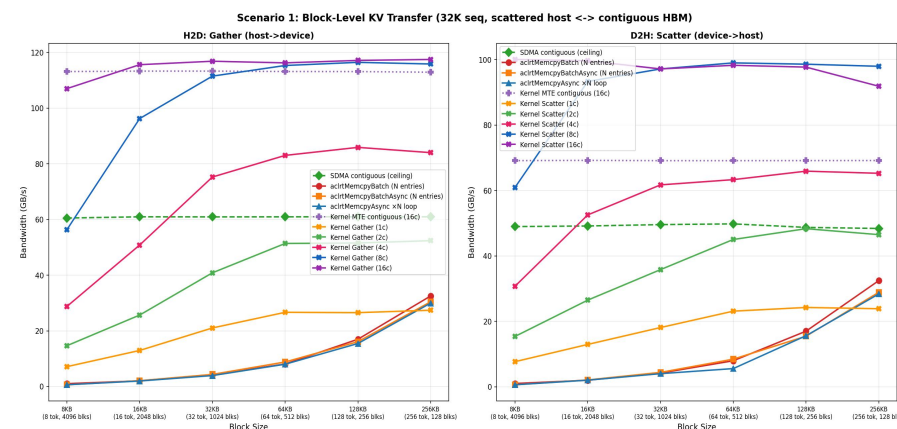
Table 1: KV access pattern categories, HBM buffer strategies, and offload characteristics

Category	Access	HBM Buffer	Growth	Examples
A: Dense	All tokens	Full buffer	$O(L)$	MHA, MLA, GQA
B: Windowed	Recent W	Rolling, size W	$O(1)$	SWA [30]
C: Compressed	All, rate $1/n$	Full, size L/n	$O(L/n)$	NSA comp. [37]
D: Recurrent	Fixed state	Current state	$O(1)$	DeltaNet [40], GDN [42]
E: Selective	Top- K	Metadata ($H_{meta} \cdot L$) + cache ($m \times K$)	$O(H_{meta} \cdot L)$	DSA [22], ShadowKV [34]

不同的Attention类型的KV增长和访问特征不同，最优KV buffer放置



基于Access类别的不同KV管理策略



基于UB的Kernel数据传输，带宽是传统方式的3倍

问题三 | PagedAttention 的非连续块寻址，是KV管理的必然形态，还是一笔可以偿还的历史债？

03

两阶段虚拟地址管理

■ 关键技术： 两步式虚拟地址管理

连续 VA → 更简单的算子、更快的 Attention、无 PagedAttention 开销

打破vLLM的第一性原理：PageAttention不是必须的

→ 第一步 Reserve — 为每个请求分配连续 VA 空间（不消耗物理显存）

→ 第二步 Bind — 按实际序列长度动态绑定 2 MB 物理页

→ SWA Unbind — 滑窗外的物理页释放回池

→ 前缀共享 — 相同前缀的请求绑定同一组物理页

→ DBO 重叠 — VA 绑定操作隐藏在当前 batch 的生成步中

→ 兼容现有算子 — 静态block table + slot mapping模拟

PagedAttention（基线）

- 每次访问都需要 Block Table 间接寻址
- 每个 FA 算子都必须实现 paged 模式

Omni Cache（连续 VA）

- 每请求每层仅需 (base_ptr, seq_len)
- 硬件 MMU 透明完成 VA → PA 翻译
- 基于HBM Pool的On-demand动态绑定，layze释放
- 可实现最高 1.99x 吞吐提升（vAttention 结论）

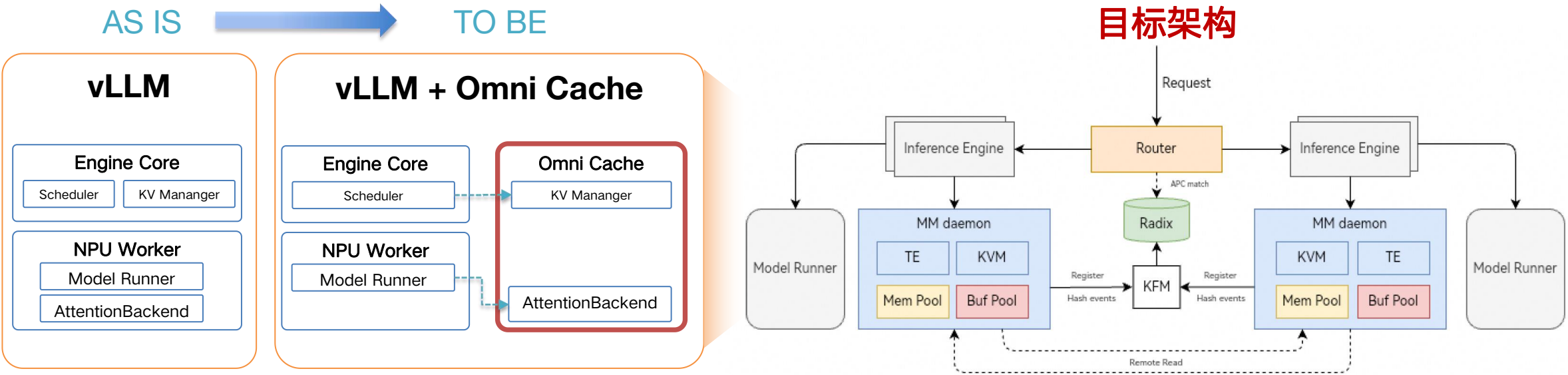
核心收益：降低 TPOT — 更简单的 Attention 元数据构建 + 连续 VA 加速 Attention 计算

问题四 | KV Cache 的生命周期是否必须绑定到推理引擎进程？

04

基于持久化大页的KVM服务

Omni Cache 系统架构：以内存+KV Fabric重构推理范式



独立内存管理MM Daemon

- 基于hugetbl fs预分配持久化内存
- 映射到NPU供算子直接访问
- 注册到RDMA网卡供其它节点单边访问
- 推理引擎重启KV不丢失
- 统一管理HBM的2MB物理池和VA

KV Fabric 跨实例跨机共享

- KV Fabric Manager统一管理MM Daemons
- Block全局统一编址：uid + block id
- KVF提供MM路由信息
- KVF统一管理Block hash

全局APC调度

- 基于集中Radix Tree的APC调度
- Inference Engine免APC匹配
- 基于统一编址实现请求级KV预加载

问题五 | 把 host memory 统一编址成全局 KV 池，能否一次性收敛 KV 共享的所有挑战？

05

KF Fabric全局共享

KV Fabric: 解耦 KV 管理与全局共享

KV 管理从推理引擎独立 → 被动 RDMA 访问 → 全局 KV 复用

解耦 KV 管理器

- KV Block Pool 管理从 vLLM 中抽离, 由独立 MM 守护进程负责
- 每个 block 分配全局 ID: $(mm_uid \ll 48) | local_block_index$
- 推理引擎仅持有临时 HBM 缓冲区 — 不拥有 KV 数据
- MM 守护进程生命周期独立于推理: 重启、换模型不影响 KV 池

RDMA 单边内存访问

- 每个 MM 启动时将 hugetlbfs 内存池注册到本地 RoCE NIC
- Fabric 内任意成员可远程读取 — 无需数据所有者 CPU 参与
- 无需推拉协调: 请求方凭全局 Block ID 直接 RDMA 读取, Omni Proxy 提前锁定

Omni Proxy/Router: 全局调度编排

- 收集所有 MM 的 block hash 事件
- 维护跨节点全局 Radix Forest
- 将请求路由到前缀匹配最长的实例
- 锁定匹配 block 防止请求到达前被淘汰
- 结果: 集群范围内最大化 KV 复用

跨节点 APC 请求流程

- Omni Proxy 对请求分词、计算前缀 hash
- 遍历 Radix Tree → 在远端节点找到匹配
- 携带全局 Block ID 路由到 decode 实例
- Decode 侧 MM 通过 RDMA 读取匹配 KV
- Decode 不保存 prefill 的 KV, APC 匹配提供混合 block ids

Host 内存池 → 所有节点统一 KV Fabric · 全局 Block ID · 单边 RDMA 无需所有者参与 · APC 命中率提升 2x

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 8月 📍 深圳 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海 🧑 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

AiCon

全球人工智能开发与应用大会

深圳站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 8 月 21-22 日 / 深圳湾万丽酒店



参会咨询

专题: Harness Engineering: 模型之外的智能体工程

专题出品人: 柴思远

智谱 / 副总裁



专题: 超级个体与蜂群智能的共生进化

专题出品人: 张昊阳

EvoMap / 创始人



专题: AI Agent 高价值商业场景实战

专题出品人: 李响 博士

平安科技 / 智能两核 AI 部总经理



专题: 超级 App 的 AI 原生化: 端智能、端云协同与用户体验重构

专题出品人: 黎三平

京东 / 端技术负责人



专题: AI 原生新范式: Coding Agent 重构软件研发全流程

专题出品人: 林香鑫

网易游戏 / 高级技术经理



专题: 智能体时代的数据供给与治理

专题出品人: 陈廷梁 (王赟)

数新智能 / 创始人兼 CEO



性能收益总结

06

Omini Cache 性能收益

核心收益总结

更长单次推理长度

释放 HBM 给激活值 + 全量KV 留 Host

- HBM KV 预留: 15 GB → 5 GB, 释放 10 GB 给激活值
- 最大 batched token 数: 32K → 105K (3×), NPU利用率提升10~20%
- 稀疏注意力: 大部分 KV 留在 Host, 释放高达80%
- Sparse Decode 并发: 最大4x

更低 TTFT

APC 命中率大幅提升

- KV 容量提升10~50x
- Host 池: 1 TB 被所有 TP rank 和实例共享
- KV Fabric 跨节点: APC 命中率提升 >2x
- 热重启: 大页文件崩溃后持久保留

更低 TPOT

更简单的元数据 + 连续 VA

- 消除 Block Table 间接寻址和 Slot Mapping
- 请求元数据简化为 (base_ptr, seq_len) / 层
- 连续 VA: Attention 吞吐最高提升 1.99x
- 硬件 MMU 翻译 — 零软件开销

HBM 内存分配对比 (V4) — 每芯片 64 GB

组件	基线	Omni Cache
系统 + 模型权重 (INT8)	37 GB	37 GB
KV 预留	15 GB (HBM)	5 GB (HBM 缓冲区)
可用激活显存	12 GB	22 GB
KV Block 数量	80,000	329,000 (Host)
最大 Batch Token 数	64K	105K
NPU 利用率	48%	61%

■ 现网性能表现

GLM 5.1

3×

Prefill Token
吞吐量提升

Up to 4×

Decode
并发提升 DSA

50×

KV Cache容量提升
(1TB内存)

Pangu V2

92B

单机PD分离支持512K长
序列, APC命中率80%

505B

W8C16单节点组P支持
512K, 吞吐提升4x

Up to 4×

Decode 并发提升

THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会